

2^η εργασία

Σκοπός της εργασίας είναι η δημιουργία Android ή iOS εφαρμογής με το AR Foundation του Unity.

Στην αρχή η εφαρμογή εμφανίζει το εξής menu / on screen buttons

- Initialize cloud anchors (δείτε πιο κάτω)
- Game

Με βάση on screen buttons ο χρήστης θα μπορεί να κάνει όταν εισέλθει στο game τα εξής

- Place object: με χρήση touch και ray casting/hit-test σε επίπεδα που έχουν ανιχνευτεί. Σε κάθε touch τοποθετείται και ένα (το ίδιο) 3D αντικείμενο
- Scale object: Touch για επιλογή του αντικειμένου, χρήση slider ή απλά της κίνησης του δαχτύλου στην οθόνη για σμίκρυνση / μεγέθυνση του αντικειμένου
- Start game (αφού τοποθετηθούν και διαμορφωθούν τα αντικείμενα)

Στη διάρκεια του παιχνιδιού ο χρήστης (με το πάτημα on-screen button) ρίχνει σφαίρες που κινούνται με physics από ένα σημείο (δείτε πχ <https://github.com/googlesamples/arcore-depth-lab#collider> για το πως μπορεί να γίνει αυτό) με σκοπό να χτυπήσει τα αντικείμενα που έχει βάλει. Σε κάθε collision με αντικείμενο αυξάνει το score κατά 1. Το score δείχνεται στο scoreboard πάνω στην εικόνα που θα αναφερθεί πιο κάτω. Αν δεν μπορείτε να το κάνετε, εμφανίστε το score κάπου στην οθόνη. Το παιχνίδι τελειώνει με αντίστοιχο κουμπί που υπάρχει στην οθόνη.

Αν η εφαρμογή ανιχνεύσει (image tracking) στη διάρκεια του παιχνιδιού κάποια από 2 εικόνες που εσείς θα επιλέξετε (τις συμπεριλαμβάνετε στο παραδοτέο για να μπορώ να τις δοκιμάσω) τότε στην μία προβάλλει ένα κάποιο βίντεο ή (extra points) το βίντεο της μπροστά κάμερας, ενώ στην άλλη το score.

Στο παιχνίδι να εμφανίζονται 2 cloud anchors (<https://developers.google.com/ar/develop/cloud-anchors#unity-ar-foundation>). Για την χρήση cloud anchors θα πρέπει να εγκαταστήσετε τα ARCore Extensions for AR Foundation. Η αρχικοποίηση (hosting) των cloud anchors θα γίνεται από το αρχικό μενού (Initialize cloud anchors). Όταν κατά την εκτέλεση του παιχνιδιού ανιχνευθεί (resolve) το 1^ο anchor (τοποθετημένο σε οριζόντια επιφάνεια) να εμφανίζεται ένα animated 3D model (πχ από το asset store) ή (extra points) ένα particle system (έτοιμο από το Unity) που τα σωματίδια να αναπηδούν στο επίπεδο (plane detection) ενώ στο 2^ο anchor ένα γυαλιστερό αντικείμενο που η επιφάνειά του παίρνει υφή με environment probe.

Τα 3D αντικείμενα θα πρέπει να φωτίζονται ρεαλιστικά με βάση τον φυσικό φωτισμό και να ρίχνουν σκιές στα επίπεδα όπου βρίσκονται (ίσως χρειαστεί <https://github.com/Unity-Technologies/arfoundation-demos#universal-render-pipeline-shader-graph-shaders>). Επίσης θα πρέπει να αποκρύπτονται αν βρίσκονται πίσω (σε σχέση με την κάμερα) από φυσικά αντικείμενα με χρήση του βάθους (depth API <https://developers.google.com/ar/develop/unity-arf/depth/developer-guide?authuser=0> <https://github.com/googlesamples/arcore-depth-lab> σε συσκευές που το υποστηρίζουν) Αν η συσκευή στην οποία τρέχει η εφαρμογή δεν υποστηρίζει depth API, απλά δεν θα υπάρχει αυτή η απόκρυψη.

Αν στη διάρκεια του παιχνιδιού ανιχνευτεί πρόσωπο (με την πίσω κάμερα) να εμφανίζεται μια face mask με εικόνα της επιλογής σας ή (extra points) κάποιο 3D αντικείμενο (π.χ μύτη στην αντίστοιχη θέση). Αν το παραπάνω δεν μπορεί για τεχνικούς λόγους (“In some implementations, face tracking requires a different camera -for example, front-facing vs rear-facing- and might be incompatible with other features”) να γίνεται μαζί με τα υπόλοιπα στοιχεία του παιχνιδιού να γίνει σαν επιμέρους standalone κομμάτι της εφαρμογής (επιλογή από το αρχικό μενού).

Για επιπλέον μονάδες το παραπάνω απλό παιχνίδι να υλοποιηθεί όχι με planes αλλά πάνω στο mesh (surface) που δίνει το depth API ώστε πχ οι σφαίρες να χτυπάνε όχι στα επίπεδα και στα 3D objects αλλά στο mesh του φυσικού χώρου και στα 3D objects. (<https://github.com/googlesamples/arcore-depth-lab#collider> απαιτεί συσκευή που να υποστηρίζει το depth api <https://developers.google.com/ar/devices>)

Επιπλέον μονάδες μπορούν να αποκτηθούν εάν ενσωματωθούν τα scene semantics (<https://developers.google.com/ar/develop/scene-semantics>), για συσκευές που το υποστηρίζουν (αυτές που υποστηρίζουν και το depth api). Για να το υλοποιήσετε προσθέστε μια ακόμη επιλογή (scene semantics) στο αρχικό μενού που όταν ενεργοποιηθεί παρουσιάζει στο χρήστη το feed της πίσω κάμερας όπου έχει υπερτεθεί, με κάποια χρώματα, το αποτέλεσμα του semantic segmentation (δουλεύει μόνο σε εξωτερικούς χώρους).

Οι προδιαγραφές της εργασίας δεν είναι αυστηρές. Αν δεν μπορείτε να υλοποιήσετε κάτι κάντε τα υπόλοιπα. Αν δεν μπορείτε να υλοποιήσετε τα cloud anchors προσπαθήστε να χρησιμοποιήσετε το geospatial api (<https://developers.google.com/ar/develop/geospatial> <https://developers.google.com/ar/develop/java/geospatial/codelab?hl=en#0>) για να θέσετε ένα οποιοδήποτε 3D μοντέλο στη γωνία Εθν. Αμύνης και Αγ. Δημητρίου Για την υλοποίηση, προσθέστε ένα button “geospatial api” στο αρχικό μενού για να ανοίγει η σχετική υπο-εφαρμογή, όπου εάν ο χρήστης είναι στο συγκεκριμένο σημείο θα βλέπει στο feed της κάμερας να έχει προστεθεί το μοντέλο. Αν δεν καταφέρετε ούτε αυτό, τοποθετήστε τουλάχιστον τα αντικείμενα που αναφέρονται στο σχετικό εδάφιο σε (local) anchors, Επίσης αν μπορείτε να ενοποιήσετε καλύτερα, σε άλλο δικό σας «σενάριο» τις διάφορες AR δυνατότητες/functionalities που περιγράψω μπορείτε να το κάνετε.

Χρήσιμα

<https://codelabs.developers.google.com/?category=ar>

<https://developers.google.com/ar> (δείτε για κάθε επιμέρους θέμα, πχ hit test, augmented images, lighting estimation, τα guides για AR foundation).

https://play.google.com/store/apps/details?id=com.google.ar.unity.arcore_depth_lab (για να δείτε τις δυνατότητες του depth API στην έτοιμη εφαρμογή depth lab, αν το υποστηρίζει η συσκευή σας).

<https://github.com/dilmerv/UnityARFoundationEssentials> και τα σχετικά YouTube Videos

Ομάδες των 2 ατόμων ή ατομικά, θα ακολουθήσει προφορική εξέταση. Η εργασία συμμετέχει κατά 4 μονάδες (40%) στον τελικό βαθμό.

Μέγιστο προσθετικών μονάδων 3

2nd Assignment

The purpose of the assignment is to create an Android or iOS application with AR Unity Foundation.

On startup, the application displays the following menu / on screen buttons

- Initialize cloud anchors (see below)
- Game

By using also screen buttons, the user shall be able to do the following when entering the game:

- Place object : on detected planes , using touch and ray casting / hit - test. At each touch a 3 D object (always the same) is placed
- Scale object : Touch the object to select it, then use a slider (or simply move your finger on the screen) to increase/decrease its size
- Start game (once the objects have been placed and scaled)

During the game, the user throws balls (by pressing an on - screen button) from a point (see e.g. <https://github.com/googlesamples/arcore-depth-lab#collider> for an example) with the aim of hitting the objects she/he has placed. The balls move with physics. For each collision with an object, the score increases by 1. The score is displayed on a scoreboard, on the image that will be mentioned below. If you cannot implement this, display the score somewhere on the screen. The game ends by pressing a button on the screen.

If the application detects (image tracking) during the game, one of the 2 images that you will choose (include them in the deliverable so I can test them), then one of them displays a video or (extra points) the video feed from the front camera, while the other shows the score .

Two cloud anchors shall appear in the game (<https://developers.google.com/ar/develop/cloud-anchors#unity-ar-foundation>) To use cloud anchors you will need to install ARCore Extensions for AR Foundation . Cloud anchors hosting will be done from the initial menu (Initialize cloud anchors button). When the 1st anchor (placed on a horizontal surface) is detected (resolved) during the game execution the app shall display an animated 3 D model (e.g. from the Unity asset store) or (extra points) a particle system (readily available in Unity) whose particles bounce on the plane (plane detection). In the 2nd anchor a shiny object whose surface is textured with environment probe shall appear.

3D objects should be realistically lit based on natural lighting. They shall cast shadows on the planes they are on (you may check <https://github.com/Unity-Technologies/arfoundation-demos#universal-render-pipeline-shader-graph-shaders>). They should also be hidden (occluded) if they are behind real world objects using depth information (depth API <https://developers.google.com/ar/develop/unity-arf/depth/developer-guide?authuser=0> <https://github.com/googlesamples/arcore-depth-lab>, on devices that support it) If the device on which the application is running does not support depth API , then no occlusion shall occur.

If a face is detected with the rear camera during the game, a face mask with an image of your choice shall appear or (extra points) a 3D object (e.g. a nose in the corresponding position). If the above cannot be implemented along with the rest of the game elements for technical reasons (“ In some implementations, face tracking requires a different camera -for example,

front-facing vs rear-facing- and might be incompatible with other features”), implement it as a separate, standalone part of the application (selected from the main menu).

For extra points, the above simple game should be implemented on the mesh (surface) provided by the depth API. In this case, for example, bullets will not collide with planes but bounce on the mesh of the physical space. (<https://github.com/googlesamples/arcore-depth-lab#collider>) requires a device that supports depth API (<https://developers.google.com/ar/devices>)

Additional points can be obtained if you integrate scene semantics (<https://developers.google.com/ar/develop/scene-semantics>), for devices that support this functionality (those that support depth api). To implement this, add one more option/button (“scene semantics”) in the main menu which, when activated, presents the user with the rear camera feed where the result of semantic segmentation are superimposed (works only outdoors).

The assignment specifications are not strict. If you can't implement something, do the rest. If you can't implement cloud anchors try using the geospatial API (<https://developers.google.com/ar/develop/geospatial> <https://developers.google.com/ar/develop/java/geospatial/codelab?hl=en#0>) to place a 3D model of your choice on the corner of Ethnikis Amyntis and Aghiou Dimitriou streets. To implement it, add a button “ geospatial api ” in the main menu that will launch the respective sub-application. If the user is at the location mentioned above she/he will see in the camera feed the added model. If you can't implement this either, at least place the objects mentioned in the relevant paragraph on (local) anchors. Also, if you can integrate in a better way, using another "scenario" of your own, the various AR features/ functionalities described above, please do so.

Useful

<https://codelabs.developers.google.com/?category=ar>

<https://developers.google.com/ar> (see for each individual topic , e.g. hit test, augmented images, lighting estimation, the AR foundation guides).

https://play.google.com/store/apps/details?id=com.google.ar.unity.arcore_depth_lab (check the depth API functionalities in the depth lab application, if your device supports it).

<https://github.com/dilmerv/UnityARFoundationEssentials> and the related YouTube Videos

Groups of 2 people or individually, an oral examination will follow. The assignment contributes 4 points (40%) to the final grade.

Maximum bonus points: 3